SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS
COURSE CODE: CSA06

COURSE OUTCOME COVERED

CO2: Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)

Assessment Tool Weightage: 10%

QUESTIONS: 10 **Total Marks:50**

Annexure A

DEBUGGING & OPTIMISATION TASK

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q1. Debugging Selection Sort Code

Errors Identified

1. **Wrong comparison operator**
   if arr[j] > arr[min_idx]:

   - This selects the **largest** element instead of the smallest.
   - For ascending order, it should be:

   if arr[j] < arr[min_idx]:

2. **Unnecessary swapping**
   - The program swaps even when min_idx == i.
   - This causes unnecessary operations.

Root Cause

- The comparison operator (>) is incorrect for ascending sorting.
- Swapping without checking wastes time, although the algorithm still works.

Step-by-Step Debugging

1. Initialize min_idx = i.
2. Compare each remaining element with the current minimum.
3. Update min_idx if a smaller element is found.
4. Swap only if the minimum element is not already in the correct position.

Optimized Code

```
def selection_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        min_idx = i

        # Find the smallest element
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j

        # Swap only if needed
        if min_idx != i:
            arr[i], arr[min_idx] = arr[min_idx], arr[i]

    return arr
```

```
# Example
arr = [64, 25, 12, 22, 11]
print(selection_sort(arr))
```

Time Complexity

- **Best Case:** $O(n^2)$
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$

Selection Sort always performs the same number of comparisons regardless of input.

Q2. Debugging Linear Search Code

Errors Identified

1. **Wrong return value when element is found**
   return -1
   - Should return the index of the element.
2. **Wrong return statement after loop**
   return i
   - Returns the last index instead of indicating failure.

Root Cause

- The success and failure return values are interchanged.

Step-by-Step Debugging

1. Traverse every element.
2. If the key is found, return its index.
3. If the loop finishes, return `-1`.

Optimized Code

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
```

```
            return i

    return -1

# Example
arr = [10, 20, 30, 40, 50]
print(linear_search(arr, 30))
```

Time Complexity

- **Best Case:** O(1)
- **Average Case:** O(n)
- **Worst Case:** O(n)

The algorithm may need to examine every element.

Q3. Debugging Closest Pair Code

Errors Identified

1. **Incorrect initialization**
   min_dist = 0
   - No distance can be smaller than 0.
2. **Wrong comparison**
   if dist > min_dist:
   - This stores the maximum distance instead of the minimum.

Root Cause

- Initial value and comparison logic are incorrect.

Step-by-Step Debugging

1. Initialize minimum distance to infinity.
2. Compute the distance for every pair.
3. Update minimum distance when a smaller value is found.

Optimized Code

```
import math

def closest_pair(points):
    min_dist = float('inf')
```

```
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            dist = math.sqrt(
                (points[i][0] - points[j][0]) ** 2 +
                (points[i][1] - points[j][1]) ** 2
            )

            if dist < min_dist:
                min_dist = dist

    return min_dist

# Example
points = [(1,2), (3,4), (2,3), (7,8)]
print(closest_pair(points))
```

Optimization

- Only compare each pair once using `j = i + 1`.
- Avoid redundant pair comparisons.

Time Complexity

- **Best Case:** $O(n^2)$
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$

Every pair of points is checked exactly once.

Q4. Debugging Convex Hull Logic

Errors Identified

1. **Incorrect orientation formula**
   return (q[1]-p[1])*(r[0]-q[0]) - (q[0]-p[0])*(r[1]-q[1])
   - The sign is reversed.
2. **Incorrect hull condition**
   - Interior points may be treated as boundary points.

Root Cause

- Wrong orientation sign causes incorrect left/right turn detection.

Correct Orientation Function

```
def orientation(p, q, r):
    return ((q[0] - p[0]) * (r[1] - p[1]) -
            (q[1] - p[1]) * (r[0] - p[0]))
```

Convex Hull Logic

```
def is_hull_edge(points, p, q):
    side = None

    for r in points:
        if r == p or r == q:
            continue

        val = orientation(p, q, r)

        if val != 0:
            if side is None:
                side = val > 0
            elif (val > 0) != side:
                return False

    return True
```

Optimization

- Skip checking points `p` and `q`.
- Stop immediately when points appear on both sides of the line.

Time Complexity

- **Best Case:** $O(n^3)$
- **Average Case:** $O(n^3)$
- **Worst Case:** $O(n^3)$

Brute-force Convex Hull checks every point against every possible edge.

Q5. Debugging Binary Search Code

Errors Identified

1. **Wrong boundary update**
2. elif arr[mid] < key:
   high = mid - 1
   - Should move the lower boundary.
3. **Wrong boundary update**
4. else:

low = mid + 1

- Should move the upper boundary.

5. **Incorrect loop condition**
   while low < high

   - Misses the final element.
   - Should use:

   while low <= high

Root Cause

- Search boundaries are updated in the wrong direction.
- Final element is skipped because of the incorrect loop condition.

Optimized Code

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = low + (high - low) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1

# Example
arr = [10, 20, 30, 40, 50]
print(binary_search(arr, 40))
```

Optimization

- Uses:

mid = low + (high - low) // 2
to avoid integer overflow in languages with fixed-size integers.

Time Complexity

- **Best Case:** O(1) – Key found at the middle.
- **Average Case:** O(log n) – Search space is halved in each iteration.
- **Worst Case:** O(log n) – Maximum number of halvings before the search ends.